

String Matching Algorithms

STRING MATCHING OR PATTERN MATCHING

Text: “G E E K S F O R G E E K S”

0 1 2 3 4 5 6 7 8 9 10 11 12

Pattern: “G E E K”

0 1 2 3

Pattern found at index **0** and **8**

To check if a pattern is present in a given text.

Text- $T[1,2,\dots,n]$, Pattern- $P[1,2,\dots,m]$, $n > m$

Types of String Matching Algorithms

- Naive's Algorithm / Brute Force Algorithm
- Rabin-Karp Algorithm
- Knuth-Morris-Pratt Algorithm (KMP)

Naive's Algorithm / Brute Force Algorithm

Example 1:

T:	a	b	c	a	b	a	b	a	b	b	a
	1	2	3	4	5	6	7	8	9	10	11
P:	a	b	a	b							
	1	2	3	4							

$s \leftarrow 0$ to $n-m$

To Check: $P[1] = T[s+1]$

$P[2] = T[s+2]$

$P[3] = T[s+3]$

hence **$P[m] = T[s+m]$** , Valid
shift = '**s**'

T: a | b | c | a | b | a | b a b b c

P: a | b | a | b s=0 **x**

a | b | a | b s=1

a | b | a | b **x**

a	b	a	b
---	---	---	---

s=2

x

s=3 **Match** & also at s=5

Example 2:

T: a b c a b a a b c a b a c

P: a b a a

Try Yourself..!

Algorithm

Naive_String_Matching (T, P)

n ← len (t)

m ← len (p)

for s ← 0 to n-m

do

if

P[1,...,

m] =

T[s+1,..

Time Complexity: $O((n-m+1)*m)$

$T(n,m) = O(nm)$

then

Drawback: We are incrementing s by only 1.

r

i

n

t

(

Rabin-Karp Algorithm

Example 1:

T: 2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1

P: 3 1 4 1 5

$\Sigma = \{ 0,1,2,\dots,9 \}$

Solution:

Select a prime number, used to find the mod value of the given pattern which is referred as **hash value**.

Rabin Karp algorithm compares string's **hash values** rather than comparing the string itself.

$T[s+1,\dots,s+m] \bmod q$ can be incrementally computed by subtracting the higher order digit, shifting and then adding the lower order digit of the next substring. All operations are done with mod q arithmetic.

T: 2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1

P: 3 1 4 1 5

q = 13 (Randomly selected)

Radix = 10

P % q = hash value

31415 % 13 = 7

Now

23590 % 13 = 8

35902 % 13 = 9

59023 % 13 = 3

90231 % 13 = 11

02314 % 13 = 0

23141 % 13 = 1

31415 % 13 = 7 (Actual Hit)

14152 % 13 = 8

41526 % 13 = 9

15267 % 13 = 5

52673 % 13 = 10

26739 % 13 = 11

67399 % 13 = 7 (Spurious Hit)

73992 % 13 = 9

39921 % 13 = 5

$$\begin{aligned}
14152 &= [(31415 - 3 * 10^4) * 10 + 2] \bmod 13 \\
&\quad [(31415 - 30000) * 10 + 2] \bmod 13 \\
&\quad [1415 * 10 + 2] \bmod 13 \\
&\quad [14152] \bmod 13 \\
&= 8
\end{aligned}$$

Example 2:

T: 3 4 1 5 9 2 6 5 3 5 9 7 9 3
8

P: 2 6

Prime no $q = 11$

Solution:

Radix = 10

$P \% q = \text{hash value}$

$26 \% 11 = 4$

$34 \% 11 = 1$

$41 \% 11 = 8$

$15 \% 11 = 4$ (Spurious
Hit)

$59 \% 11 = 4$ (Spurious
Hit)

$92 \% 11 = 4$ (Spurious
Hit)

$26 \% 11 = 4$ (Actual Hit)

$65 \% 11 = 10$

$35 \% 11 =$
2

$58 \% 11 =$
3

$89 \% 11 =$
1

$97 \% 11 =$
9

$79 \% 11 =$
2

$93 \% 11 =$
5

$$\begin{aligned}
41 &= [(34 - 3 * 10^1) * 10 + 1] \bmod 11 \\
&= [(34 - 30) * 10 + 1] \bmod 11 \\
&= [4 * 10 + 1] \bmod 11 \\
&= [41] \bmod 11 \\
&= 8
\end{aligned}$$

Applications: Bio-informatics to see similarities of proteins.

Example 3:

T: 4 2 1 5 8 3 6 6 3 4 8 9 1 9 3 5 0 6 6 3

P: 6 6 3

Prime no $q = 13$

Apply Rabin Karp algorithm to check the presence of pattern in the given text.

Find the number of Spurious hit and exact hit.

Try Yourself...!

How to work with Strings?

Text (T): a b c d a b d

Pattern (P): a b d

$m = 3$

$d = 256$

Assume $q = 101$

The ASCII values of

$a = 97, b = 98,$

$c = 99, d = 100$

Perform Pattern hash p (for abd)

$$p = (97 \cdot 256^2 + 98 \cdot 256 + 100) \bmod 101 = 28$$

- Group the given text into strings of 3 characters and find the hash value same as above
 - (i.e.) **abc - 90, bcd - 31, cda - 56, dab - 28, abd - 28**
- Proceed with the identification of Hits as Valid and Spurious Hits
 - 2 Hits, 1 valid and 1 spurious
- Do a character wise comparison of the matched hash values

RABIN-KARP-MATCHER (T, P, d, q)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $h = d^{m-1} \bmod q$ 
4   $p = 0$ 
5   $t_0 = 0$ 
6  for  $i = 1$  to  $m$            // preprocessing
7       $p = (dp + P[i]) \bmod q$ 
8       $t_0 = (dt_0 + T[i]) \bmod q$ 
9  for  $s = 0$  to  $n - m$        // matching
10     if  $p == t_s$ 
11         if  $P[1..m] = T[s+1..s+m]$ 
12             print "Pattern occurs with shift"  $s$ 
13     if  $s < n - m$ 
14          $t_{s+1} = (d(t_s - T[s+1])h) + T[s+m+1]) \bmod q$ 
```

Time complexity : Worst Case

$$\begin{aligned} T(n, m) &= O(m) + \\ &O((n-m+1)*m) \\ &= O(mn) \end{aligned}$$

Avg Case

$$\begin{aligned} T(n, m) &= O(m) + O(n-m+1) = \\ &O(m+n) \end{aligned}$$

Knuth-Morris-Pratt Algorithm (KMP)

Consider an example

i	1	2	3	4	5	6	7	8	9	10
T:	a	a	a	a	a	a	a	a	a	b
j	1	2	3	4	5					
P:	a	a	a	a	b					

With Naive algorithm, more number of comparisons.

KMP algorithm two components

1. Prefix Function π - Compares P with itself
 2. KMP Matcher – Compares text T with Pattern P and uses π
- A prefix function π gives the information about the pattern “P” matches against shifts of itself.
 - $\pi[q]$ is the length of the longest prefix of “P” that is a proper suffix of P_q

Given the pattern P, compute prefix function.

1

.

q	1	2	3	4	5
P[q]	b	a	b	a	b
$\pi[q]$	0	0	1	2	3

2

.

q	1	2	3	4	5
P[q]	a	b	a	b	b
$\pi[q]$	0	0	1	2	0

3

•

q	1	2	3	4	5	6	7	8	9	10
P[q]	a	b	a	b	a	b	a	b	c	a
$\pi[q]$	0	0	1	2	3	4	5	6	0	1

4

•

q	1	2	3	4	5	6	7
P[q]	a	b	a	b	a	c	a
$\pi[q]$	0	0	1	2	3	0	1

5. P = b a c b c b a c a b

5. P = a c b c a c b a c d b a

Try Yourself..!

5. P = ^{1 2 3 4 5 6 7 8 9 10}
 $\boxed{b} \boxed{a} c b c b a c a b$

Step 1

Compare 2nd with 1st element
 Not Matching, hence $\pi(2) = 0$

^{1 2 3 4 5 6 7 8 9 10}
 $\boxed{b} \boxed{a} c b c b a c a b$
 0 0

^{1 2 3 4 5 6 7 8 9 10}
 $\boxed{b} a \boxed{c} b c b a c a b$ Step 2
 0 0 0

^{1 2 3 4 5 6 7 8 9 10}
 $\boxed{b} a c \boxed{b} c b a c a b$ Step 3
 0 0 0 1

^{1 2 3 4 5 6 7 8 9 10}
 $b \boxed{a} c b \boxed{c} b a c a b$ Step 4
 0 0 0 1 0

^{1 2 3 4 5 6 7 8 9 10}
 $\boxed{b} \boxed{a} \boxed{c} \boxed{b} c \boxed{b} \boxed{a} \boxed{c} \boxed{a} b$ Step 5, 6, 7, 8
 0 0 0 1 0 1 2 3 0

^{1 2 3 4 5 6 7 8 9 10}
 $\boxed{b} a c b c b a c a \boxed{b}$ Step 9
 $\pi = 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 2 \ 3 \ 0 \ 1$

COMPUTE-PREFIX-FUNCTION(P)

```
1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11 return  $\pi$ 
```

$$m = 10$$

$$\pi[1] = 0$$

$$K = 0$$

$$q = 2,$$

$$\pi[2] = 0$$

$$q = 3$$

$$\pi[3] = 0$$

$$q = 4$$

$$K = 1$$

$$\pi[4] = 1$$

$$q = 5$$

$$K = 0$$

$$\pi[5] = 0$$

$$q = 6$$

$$K = 1$$

$$\pi[6] = 1$$

$$q = 7$$

$$K = 2$$

$$\pi[7] = 2$$

$$q = 8$$

$$K = 3$$

$$\pi[8] = 3$$

$$q = 9$$

$$K = 0$$

$$\pi[9] = 0$$

$$q = 10$$

$$K = 1$$

$$\pi[10] = 1$$

1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



first comparison.

1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



1 2 3 4 5 6 7 8 9 10
b a c b c b a c a b



KMP-Matcher(T, P)

$n \leftarrow T.length$

$m \leftarrow P.length$

$\pi \leftarrow \text{Compute-Prefix-Function}(P)$

$q \leftarrow 0$

create new (empty) queue Q

for $i \leftarrow 1$ **to** n **do**

while $q > 0$ **and** $P[q + 1] \neq T[i]$ **do** $q \leftarrow \pi[q]$

if $P[q + 1] = T[i]$ **then** $q \leftarrow q + 1$

if $q = m$ **then**

 enqueue($Q, i - m$)

$q \leftarrow \pi[q]$

return Q

Example 1:

T: a b a b a b a b b

P: a b a b b

Solution:

Find the prefix values

q	1	2	3	4	5
P[q]	a	b	a	b	b
$\pi[q]$	0	0	1	2	0

$T = a b a b a b a b b$

$P = a b a b b$

$\pi[P] = 0 0 1 2 0$

$n = 9$

$m = 5$

$\pi = [0 0 1 2 0]$
 1 2 3 4 5

$q = 0, 1, 2, 3, 4$

$i = 1, 2, 3, 4, 5$

$q = 2, 3, 4$

$i = 5, 6, 7$

$q = 2, 3, 4, 5$

$i = 7, 8, 9$

1	2	3	4	5	6	7	8	9
✓	✓	✓	✓	✗				
a	b	a	b	a	b	a	b	b
↓	↓	↓	↓	↓				
a	b	a	b	b				

1	2	3	4	5	6	7	8	9
				✓	✓	✗		
a	b	a	b	a	b	a	b	b
				↙	↘	↗		
a	b	a	b	b	b			

						✓	✓	✓
1	2	3	4	5	6	7	8	9
a	b	a	b	a	b	a	b	b
						↖	↖	↖
a	b	a	b	b	b			

$s = i - m$ position.

Trace the algorithm.

T:	a	b	a	b	a	b	a	b	b
P:	a	b	a	b	b				
			a	b	a	b	b		
					b	a			

Print occurs

with 4 shift.

Analysis: $T(n,m) = O(m) + O(n+m)$
KMP matches

Applications: Intrusion Detection, DNA string search

KMP: High speed multi stream packet scanning

Naive: Keyword search in large datasets

Rabin Karp: Higher dimensional pattern matching Problems

Example 2:

T: b a c b a b a b a a a b c b a b

P: a b a b a c a

Try yourself..!